Justificação estilo Clientes-Servidor em camadas

A aplicação TaskFlow-XOps utiliza uma arquitetura cliente-servidor em camadas, constituída por três componentes principais: o frontend, a API HTTP desenvolvida em Node.js e o armazenamento temporário das tarefas em memória.

Separação de responsabilidades

Esta arquitetura permite dividir as responsabilidades entre os diferentes componentes da aplicação. O frontend, desenvolvido em HTML, CSS e JavaScript, é responsável pela interface e pela interação com o utilizador. O backend, desenvolvido em Node.js, recebe e processa os pedidos HTTP, e permite criar, listar, concluir e reabrir tarefas. O armazenamento em memória é responsável por guardar temporariamente as informações das tarefas.

Simplicidade de implementação

A utilização desta arquitetura facilita o desenvolvimento e a organização do projeto, uma vez que cada componente possui uma função específica. Além disso, a utilização dos módulos nativos do Node.js permite implementar o servidor sem recorrer a frameworks externas, tornando a aplicação mais simples e adequada ao protótipo inicial.

Facilidade de testar a API

A separação entre o frontend e o backend permite testar a API independentemente da interface gráfica. Através dos endpoints HTTP, é possível verificar se as operações sobre as tarefas funcionam corretamente. O projeto também utiliza testes automáticos executados através do comando `npm test`, que são integrados no pipeline do GitHub Actions para ajudar a identificar erros.

Limitações da arquitetura atual

Apesar de a aplicação TaskFlow-XOps cumprir os objetivos definidos para o protótipo inicial, apresenta algumas limitações que poderão ser resolvidas em futuras versões.

Ausência de persistência de dados

Atualmente, as tarefas são armazenadas num array em memória (`tasks[]`). Isto significa que, sempre que o servidor é reiniciado, as tarefas anteriormente criadas são perdidas, uma vez que não existe armazenamento permanente.

Ausência de base de dados

A aplicação ainda não possui uma base de dados para guardar as tarefas. Futuramente, poderá ser integrada uma base de dados, como MySQL ,permitindo guardar e recuperar os dados permanentemente.

Ausência de autenticação

O projeto ainda não possui um sistema de autenticação de utilizadores. Assim, não existe um mecanismo de identificação dos utilizadores nem controlo de acesso às funcionalidades da aplicação.

Ausência de deploy automático

Embora o projeto utilize GitHub Actions para realizar verificações de sintaxe e testes automáticos, ainda não existe um processo de deploy automático que permita publicar a aplicação num servidor após a validação do código.

Estas limitações representam oportunidades de melhoria para os próximos sprints, permitindo evoluir a aplicação ao nível da persistência de dados, segurança, automatização e qualidade do software.